ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 7 - Due date 03/06/26

Josh Salzberg

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github. And to do so you will need to fork our repository and link it to your RStudio.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A07_Sp26.Rmd"). Then change "Student Name" on line 4 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Sakai.

Packages needed for this assignment: "forecast","tseries". Do not forget to load them before running your script, since they are NOT default packages.\

Set up

```
#Load/install required package here
libraries <- c('forecast','tseries', 'here', 'dplyr','lubridate')
lapply(libraries,require,character.only=T)
```

```
## Loading required package: forecast

## Loading required package: tseries

## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo

## Loading required package: here

## here() starts at C:/Users/jhsal/OneDrive - Duke University/797/TSA2026/TSA_Sp26

## Loading required package: dplyr

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

## Loading required package: lubridate

##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union

## [[1]]
## [1] TRUE
##
## [[2]]
## [1] TRUE
##
## [[3]]
## [1] TRUE
##
## [[4]]
## [1] TRUE
##
## [[5]]
## [1] TRUE
```

```
here()
```

```
## [1] "C:/Users/jhsal/OneDrive - Duke University/797/TSA2026/TSA_Sp26"
```

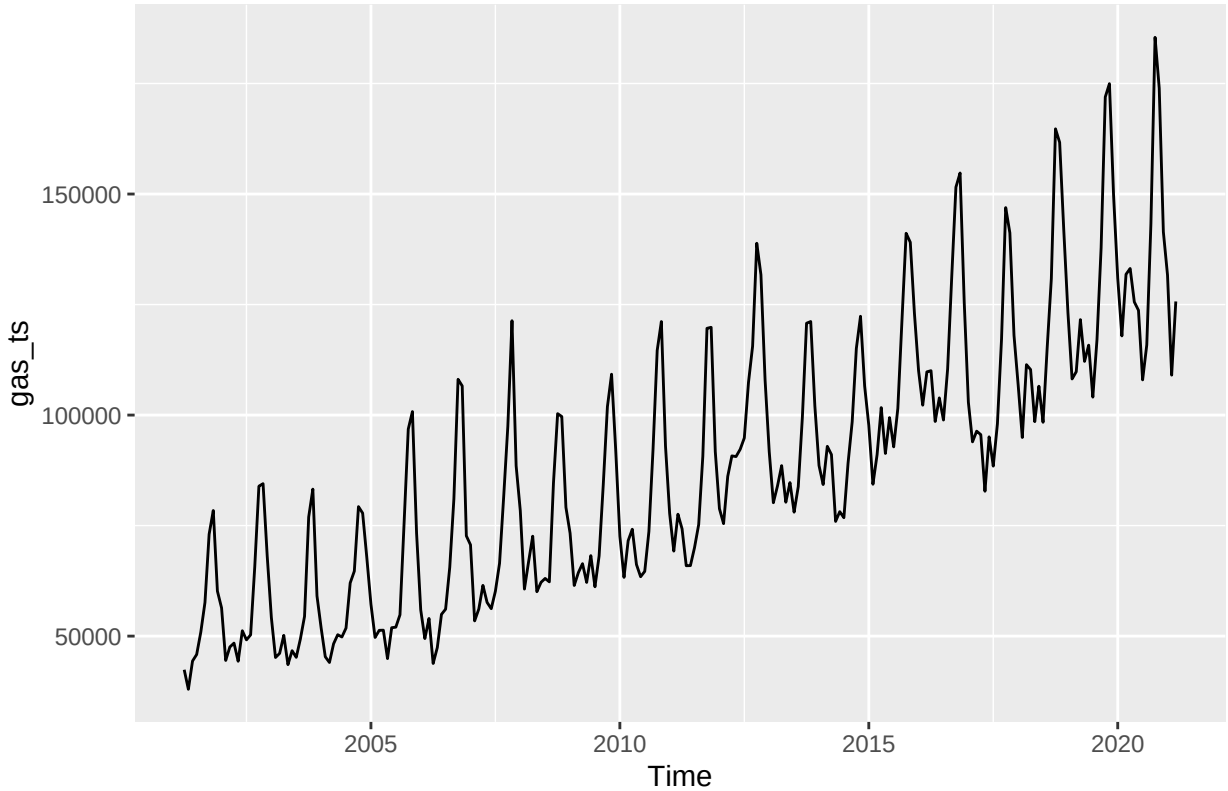
Importing and processing the data set

Consider the data from the file "Net_generation_United_States_all_sectors_monthly.csv". The data corresponds to the monthly net generation from January 2001 to December 2020 by source and is provided by the US Energy Information and Administration. **You will work with the natural gas column only**.

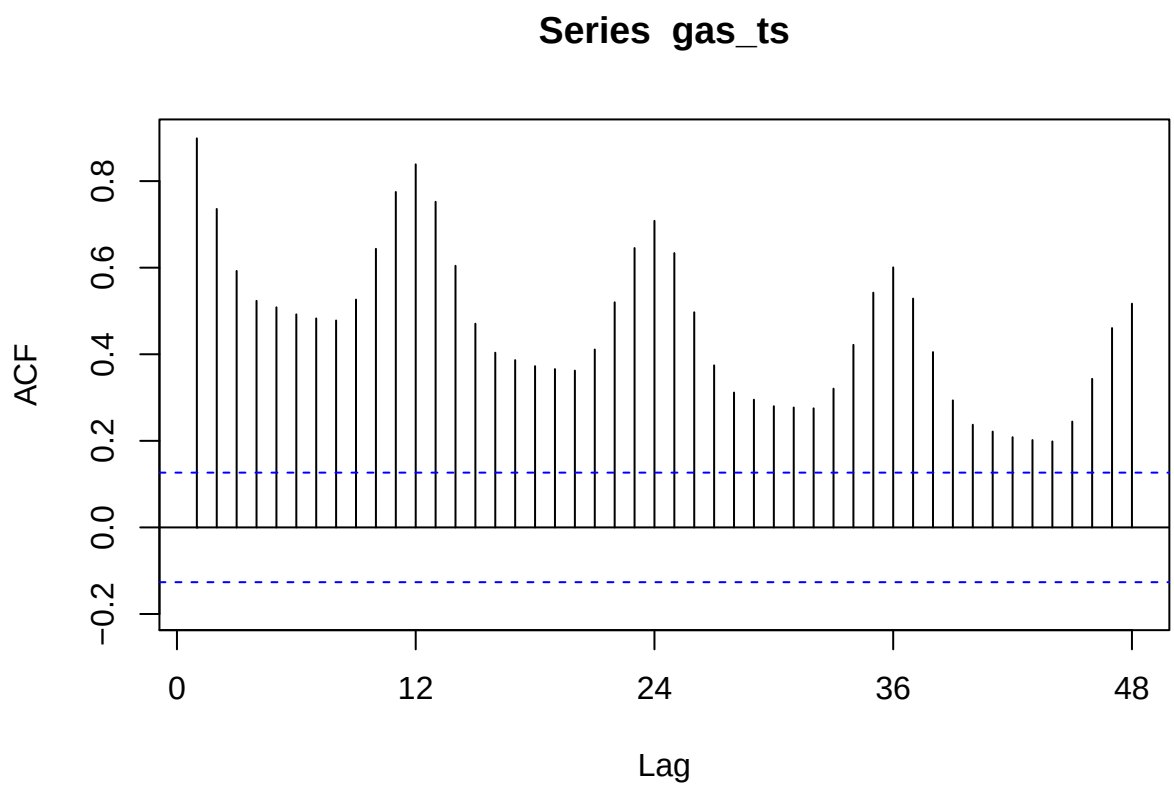
Q1

Import the csv file and create a time series object for natural gas. Make you sure you specify the **start=** and **frequency=** arguments. Plot the time series over time, ACF and PACF.

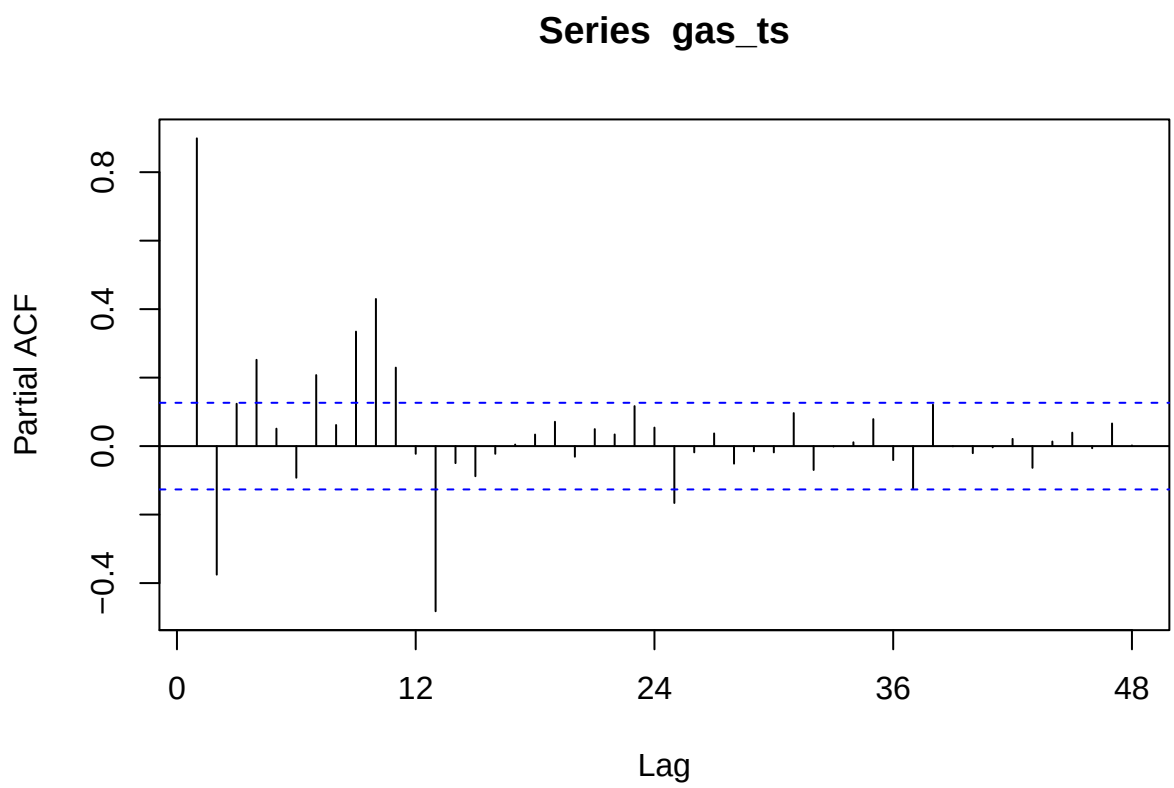
```
rawdata <- read.csv(file = here('Data',"Net_generation_United_States_all_sectors_monthly.csv"), skip = 4
gasdata <- rawdata %>%
  select(Month,natural.gas.thousand.megawatthours) %>%
  mutate(Month = my(Month)) %>%
  arrange(Month)

gas_ts <- ts(gasdata[,2], start = c(2001,4),frequency =12)

autoplot(gas_ts)
```




```r
acf_base <- autoplot(Acf(gas_ts, lag.max = 48))
```



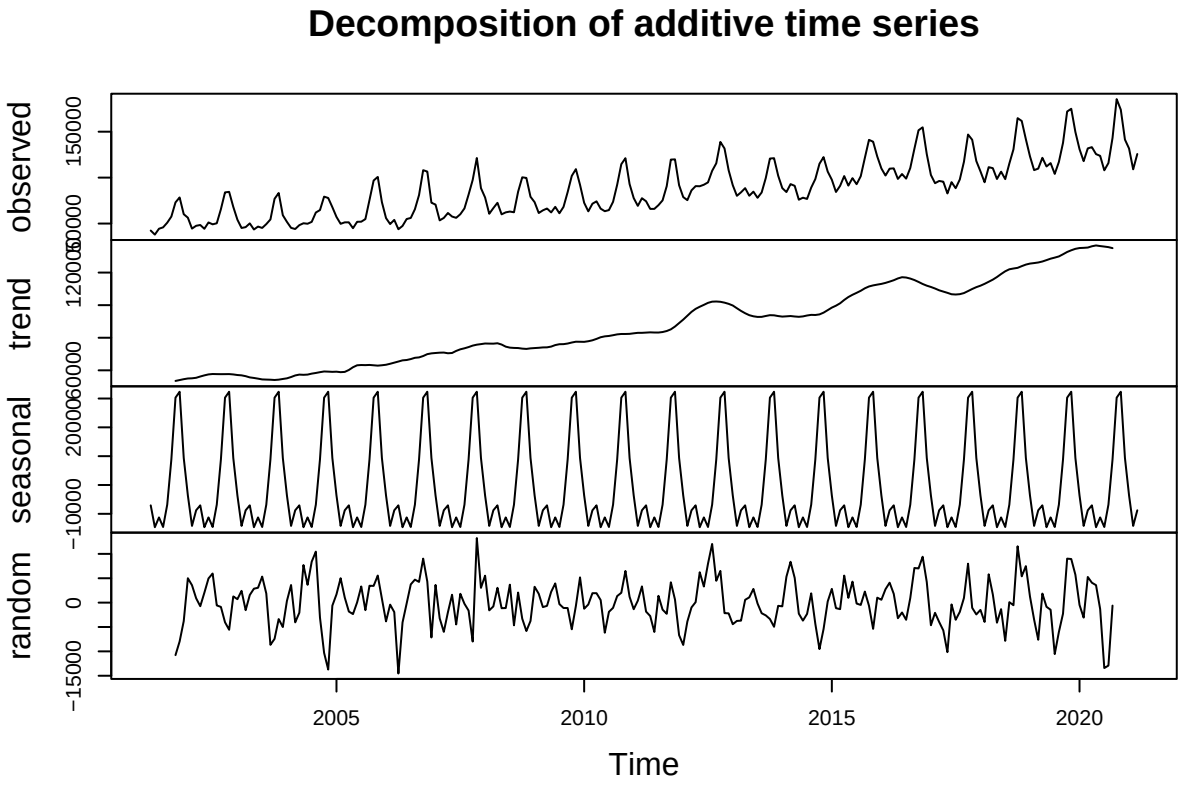

```
pacf_base <- autoplot(Pacf(gas_ts, lag.max = 48))
```



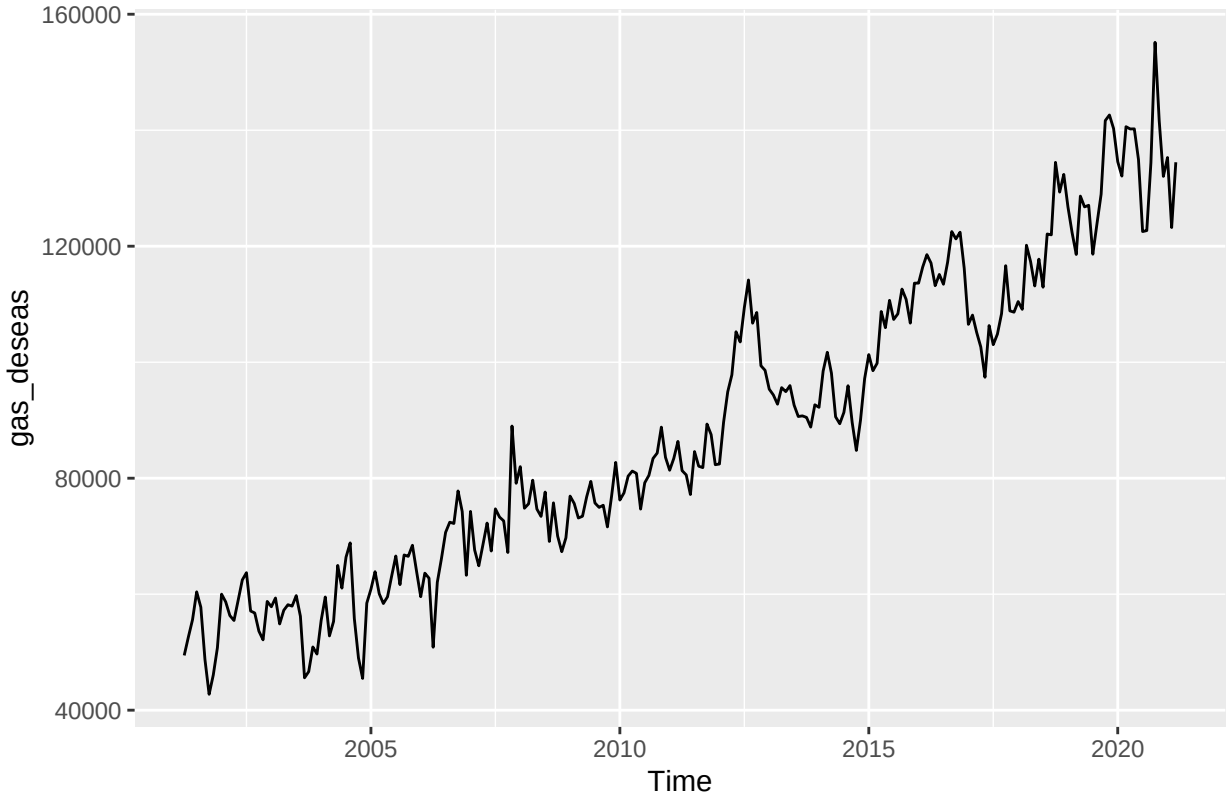
Q2

Using the *decompose*() and the *seasadj*() functions create a series without the seasonal component, i.e., a deseasonalized natural gas series. Plot the deseasonalized series over time and corresponding ACF and PACF. Compare with the plots obtained in Q1.

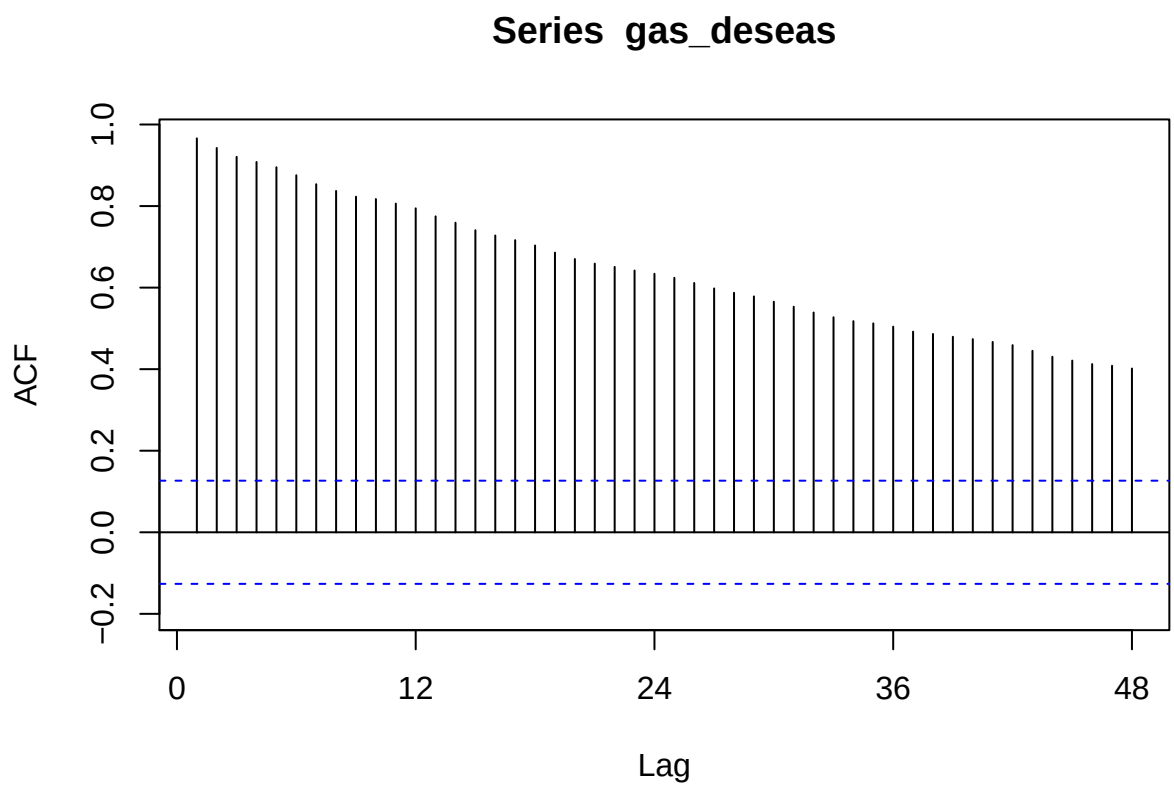
```
gas_decomp <- decompose(gas_ts)
plot(gas_decomp)
```



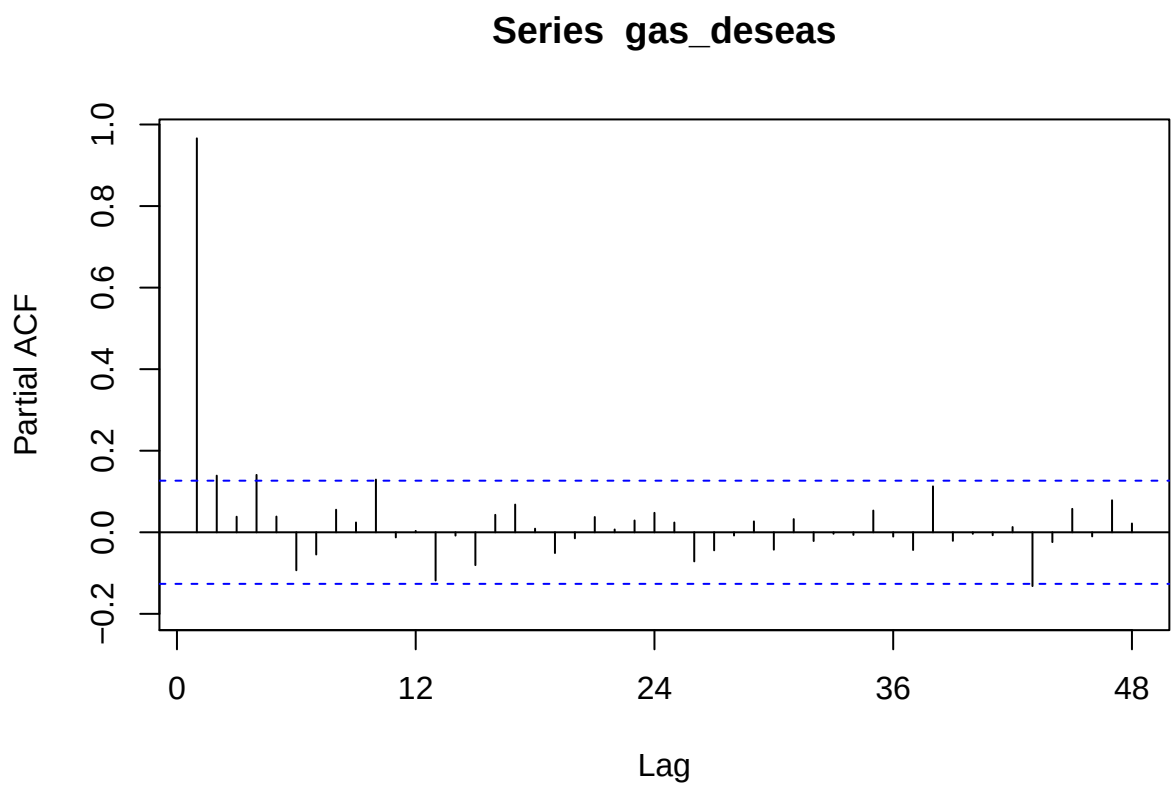

```
gas_deseas <- seasadj(gas_decomp)
autoplot(gas_deseas)
```



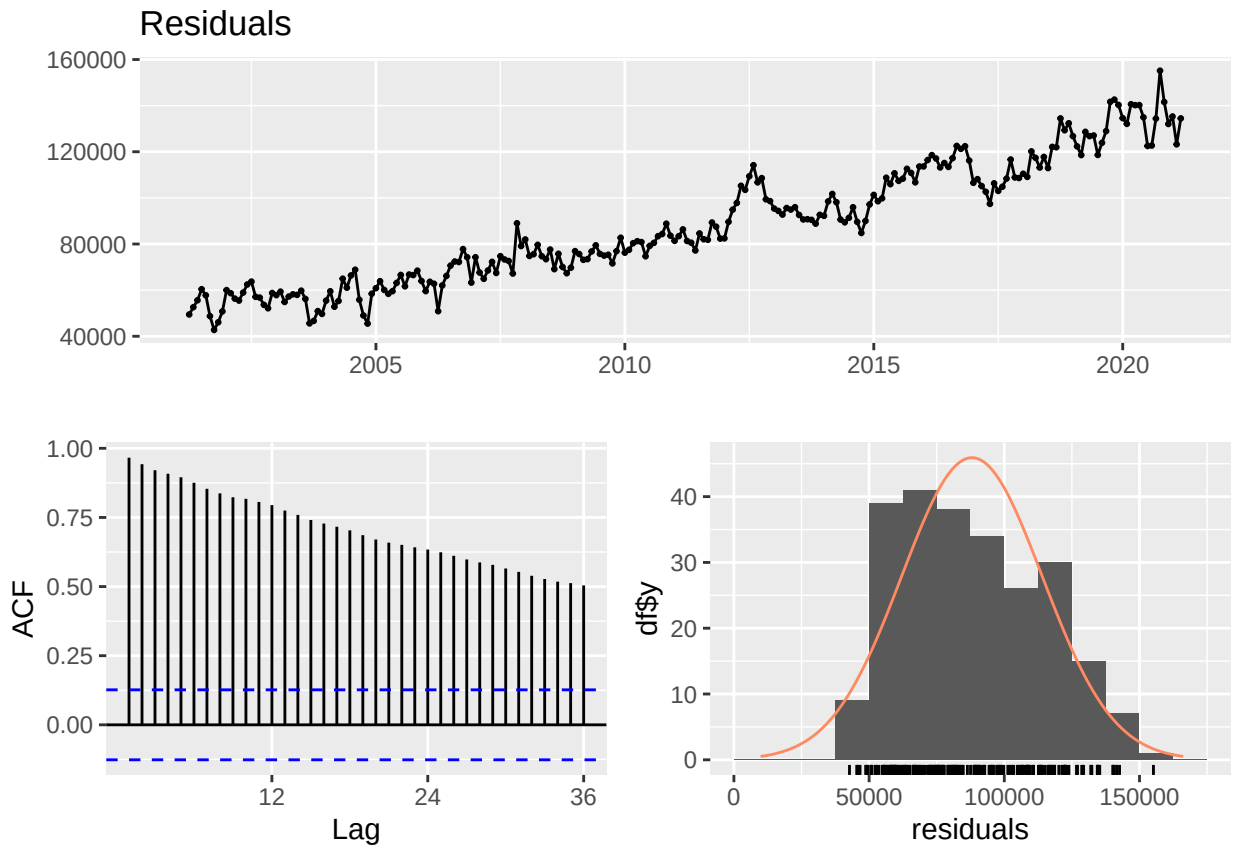

```
acf_deseas <- autoplot(Acf(gas_deseas, lag.max = 48))
```




```r
pacf_deseas <- autoplot(Pacf(gas_deseas, lag.max = 48))
```

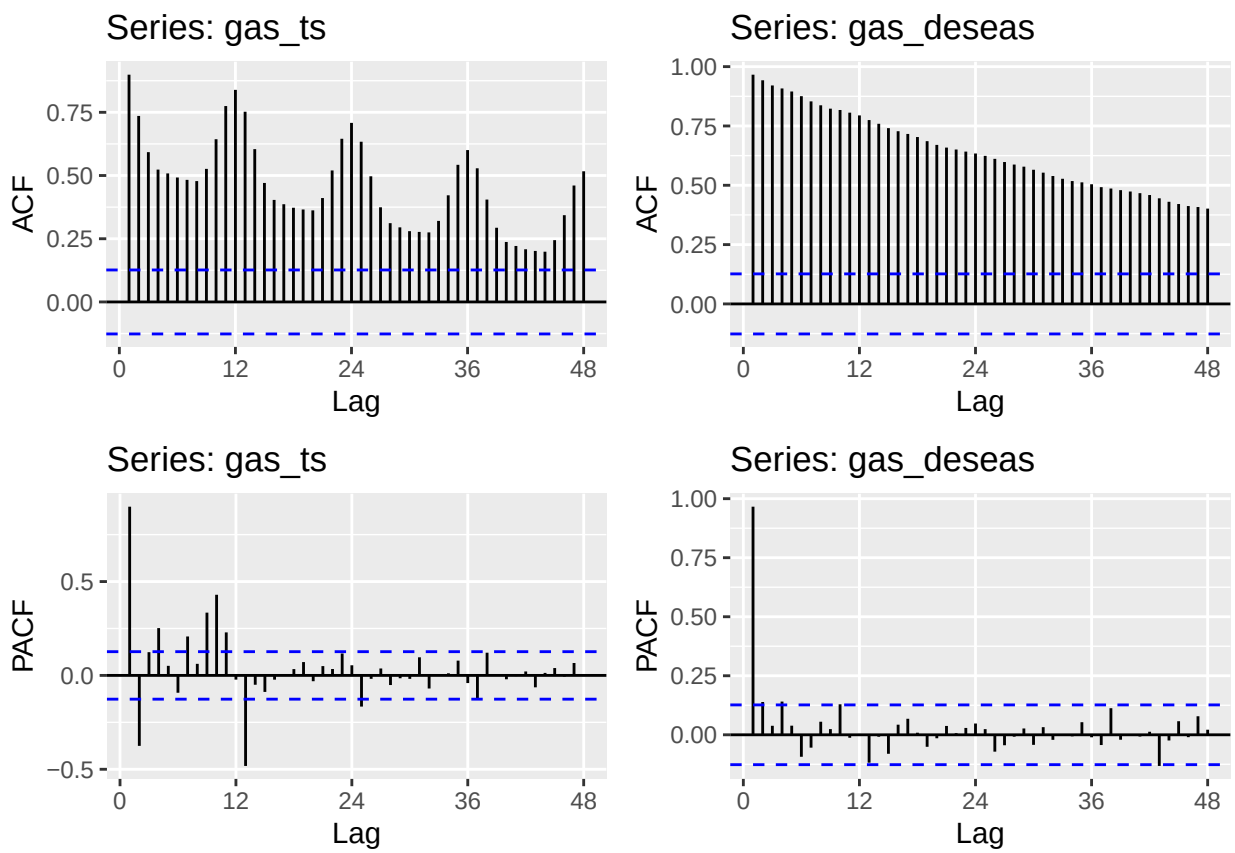



```
checkresiduals(gas_deseas)
```




```
##
##  Ljung-Box test
##
## data:  Residuals
## Q* = 3796.6, df = 24, p-value < 2.2e-16
##
## Model df: 0.   Total lags used: 24
```

```
cowplot::plot_grid(acf_base, acf_deseas, pacf_base,pacf_deseas, ncol = 2, nrow = 2)
```



Answer: We see that the ACF has transformed from strongly seasonal series to a very long decay exceeding 48 lags. The PACF on the other hand, has "cleaned up" and now demonstrates only 1 very strong lag.

Modeling the seasonally adjusted (i.e., the deseasonalized series)

Q3

Run the ADF test and Mann Kendall test on the deseasonalized data from Q2. Report and explain the results.

```
print(adf.test(gas_deseas, alternative = "stationary"))
```

```
## Warning in adf.test(gas_deseas, alternative = "stationary"): p-value smaller
## than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  gas_deseas
## Dickey-Fuller = -4.0271, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```
print(summary(Kendall::MannKendall(gas_deseas)))
```

```
## Score =  24186 , Var(Score) = 1545533
## denominator =  28680
## tau = 0.843, 2-sided pvalue =< 2.22e-16
## NULL
```

Answer: The Dickey-Fuller test statistic was -4.027 with a p-value equal to 0.01; thus we reject the null hypothesis and accept the alternative hypothesis that the series lacks a unit root. The MannKendall test returned a 2-sided p-value =< 2.22e-16. Thus we accept the alternative hypothesis that there is a statistically significant monotonic trend.

Q4

Using the plots from Q2 and test results from Q3 identify the ARIMA model parameters p, d and q. Note that in this case because you removed the seasonal component prior to identifying the model you don't need to worry about seasonal component. Clearly state your criteria and any additional function in R you might use. DO NOT use the *auto.arima*() function. You will be evaluated on ability to understand the ACF/PACF plots and interpret the test results.

```
ndiffs(gas_deseas) # supports
```

```
## [1] 1
```

Answer: 1 immediate cutoff in PACF -> assume Autoregressive component p = 1 Lacks unit root structure according to ADF test, but the persistence in AR indicates we will need d = 1 This is supported by ndiffs() function returning a 1 ACF's slow decay does not support clear MA component, so q = 0 #Propose model ARIMA(1,1,0)

Q5

Use `Arima()` from package "forecast" to fit an ARIMA model to your series considering the order estimated in Q4. You should allow constants in the model, i.e., `include.mean = TRUE` or `include.drift=TRUE`. **Print the coefficients** in your report. Hint: use the `cat()` or `print()` function to print.

```
gas_110 <- Arima(gas_deseas, order = c(1,1,0), include.drift = T)

cat("The coefficients of the ARIMA(1,1,0) model are", print(gas_110$coef))
```

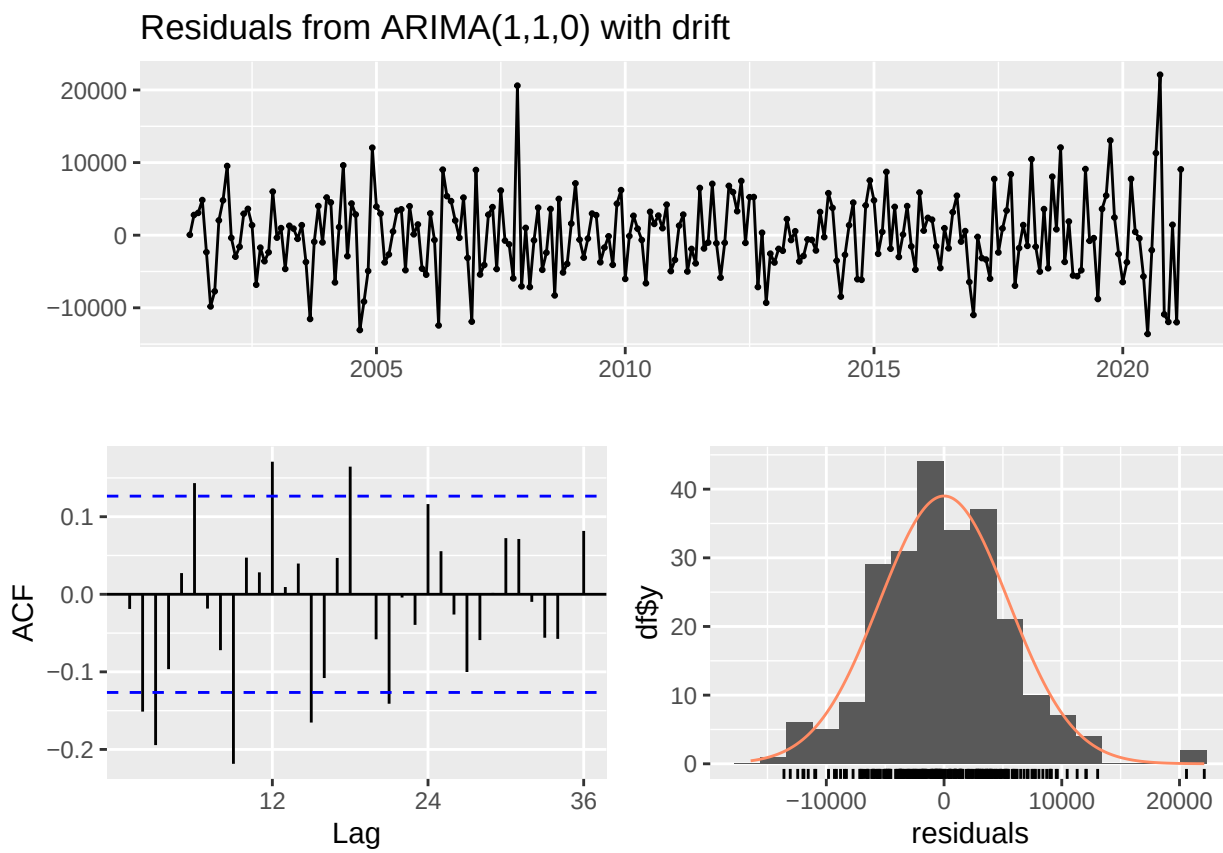
```
##         ar1       drift
##  -0.1478609 348.3926570
## The coefficients of the ARIMA(1,1,0) model are -0.1478609 348.3927
```

```
# Unfortunately the Ljung-Box score is really terrible for this model
```

Q6

Now plot the residuals of the ARIMA fit from Q5 along with residuals ACF on the same window. You may use the *checkresiduals*() function to get the plots. Do the residual series look like a white noise series? Why?

```
checkresiduals(gas_110)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,0) with drift
## Q* = 72.475, df = 23, p-value = 5.015e-07
##
## Model df: 1.   Total lags used: 24
```

answer: The residuals appear like a white noise series - the residuals cycle around the 0 line and there's no clear patterns. The ACF shows no surprisingly significant lags. It appears the residuals are *i.i.d.*

Modeling the original series (with seasonality)

Q7

Repeat Q3-Q6 for the original series (the complete series that has the seasonal component). Note that when you model the seasonal series, you need to specify the seasonal part of the ARIMA model as well, i.e., P, D and Q.

```r
print(adf.test(gas_ts, alternative = "stationary"))
```

```
## Warning in adf.test(gas_ts, alternative = "stationary"): p-value smaller than
## printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
```

```
## data:  gas_ts
## Dickey-Fuller = -8.9602, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
# No unit root present

print(summary(Kendall::MannKendall(gas_ts)))
```

```
## Score =  18658 , Var(Score) = 1545533
## denominator =  28680
## tau = 0.651, 2-sided pvalue =< 2.22e-16
## NULL
```

```r
# There is a monotonic trend

ndiffs(gas_ts)  # Returned 1
```

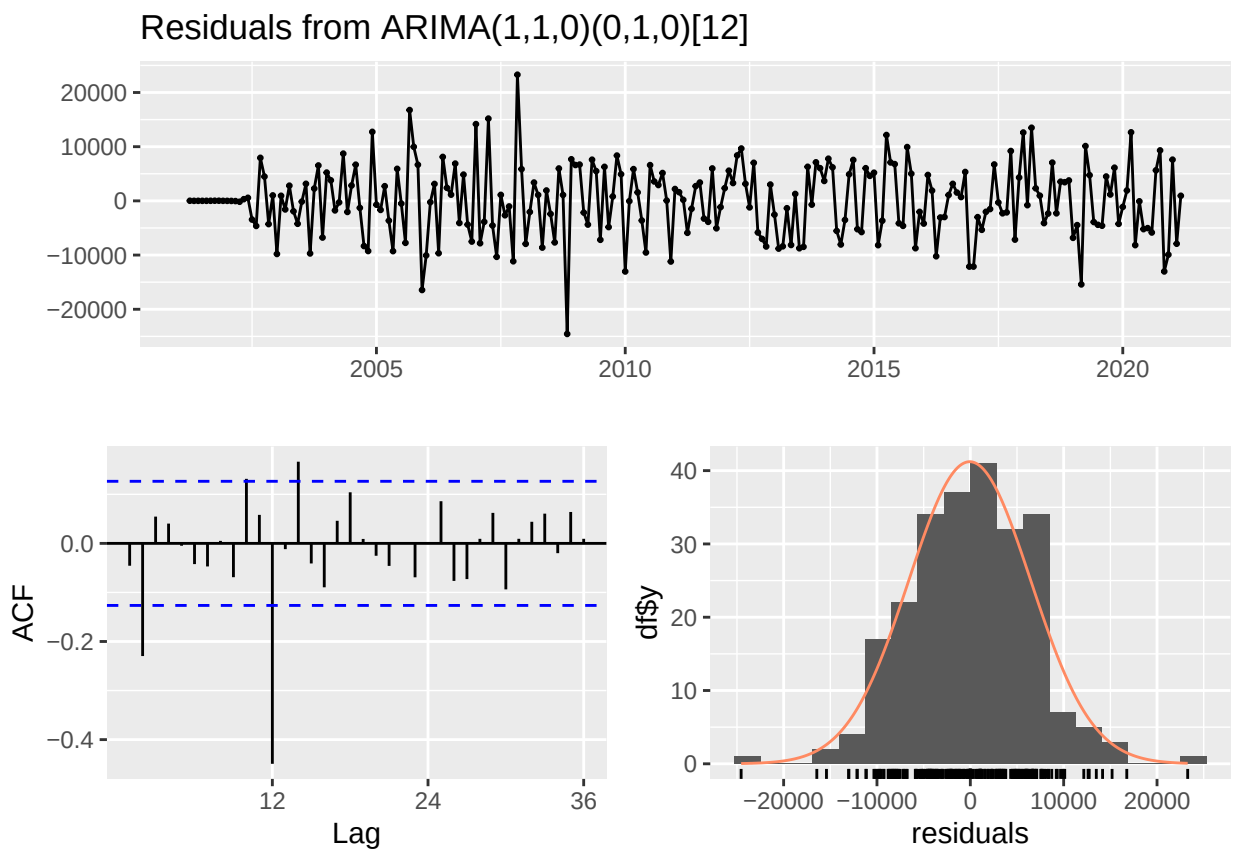
```
## [1] 1
```

```r
nsdiffs(gas_ts, frequency(12)) # returned 1
```

```
## Warning in nsdiffs(gas_ts, frequency(12)): Specified alpha value is larger than
## the maximum, setting alpha=0.1
```

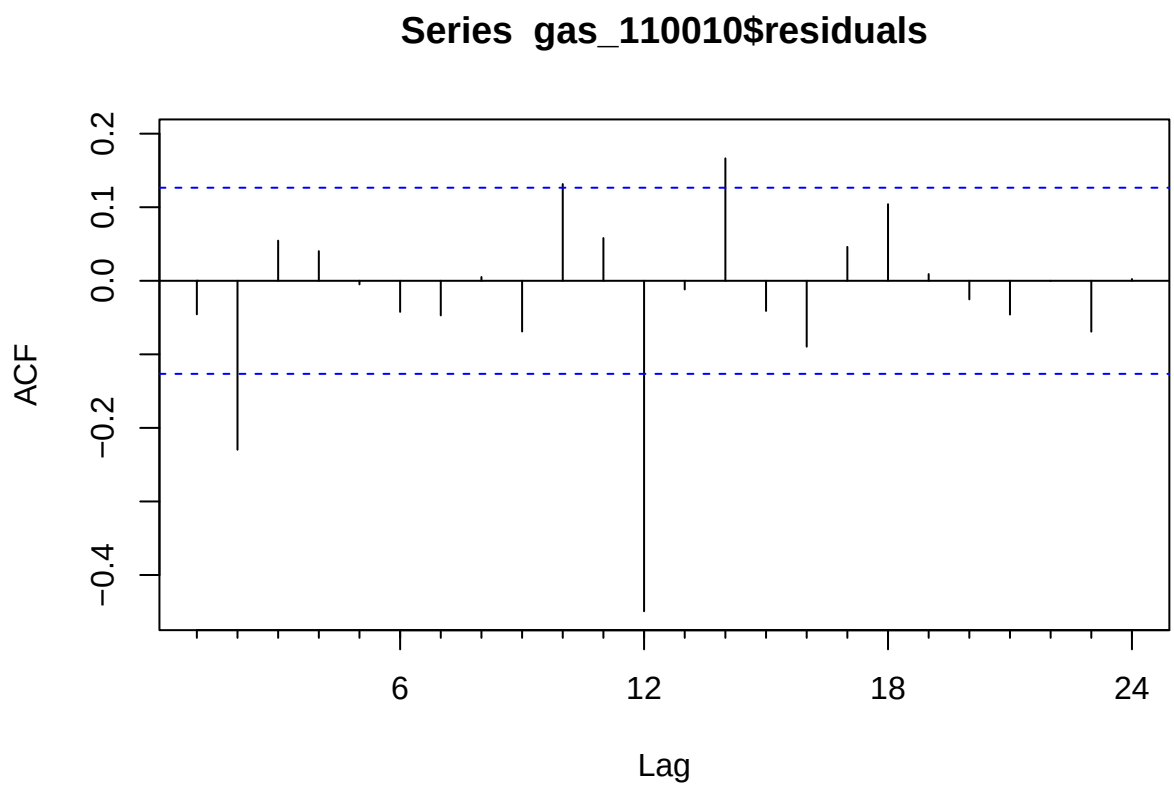
```
## [1] 1
```

```r
# We know there is a strong seasonal component, and a high persistence, so can assume both d = 1 & D =
# but uncertain the seasonal P & Q
gas_110010 <- Arima(gas_ts, order = c(1,1,0), seasonal = c(0,1,0))
checkresiduals(gas_110010)
```

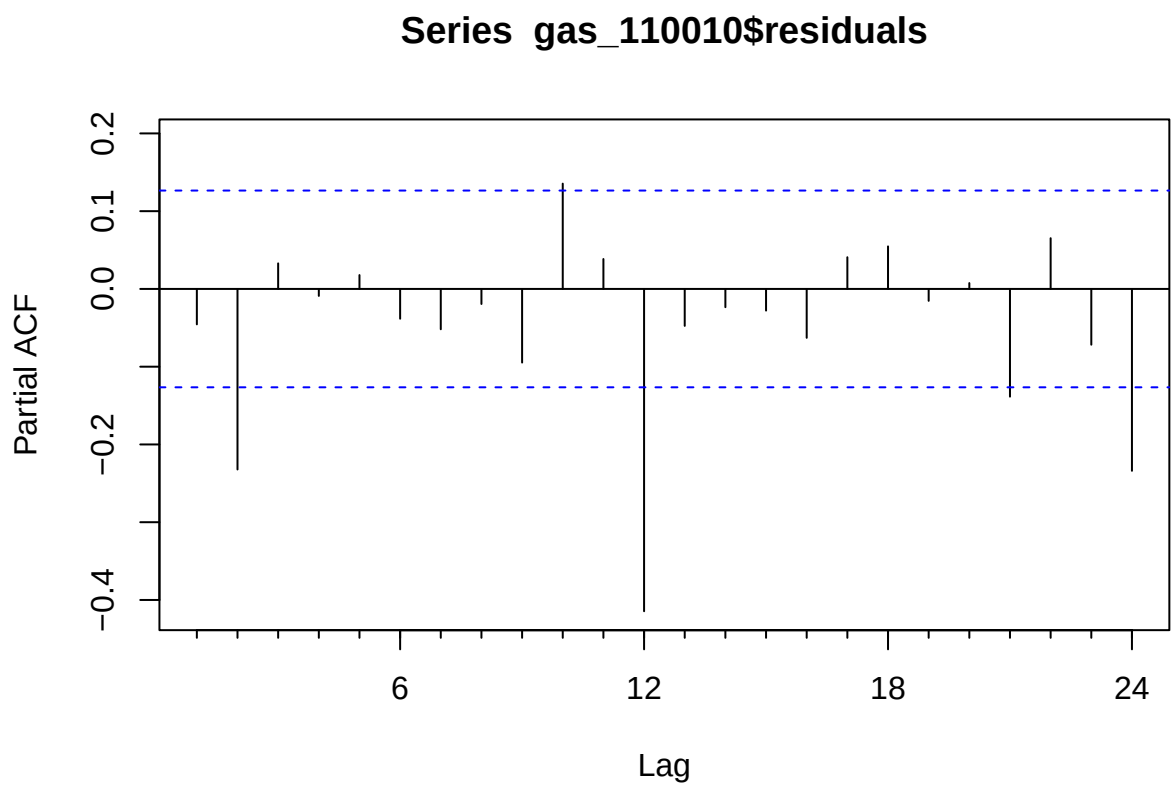



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,0)(0,1,0)[12]
## Q* = 88.423, df = 23, p-value = 1.306e-09
##
## Model df: 1.   Total lags used: 24
```

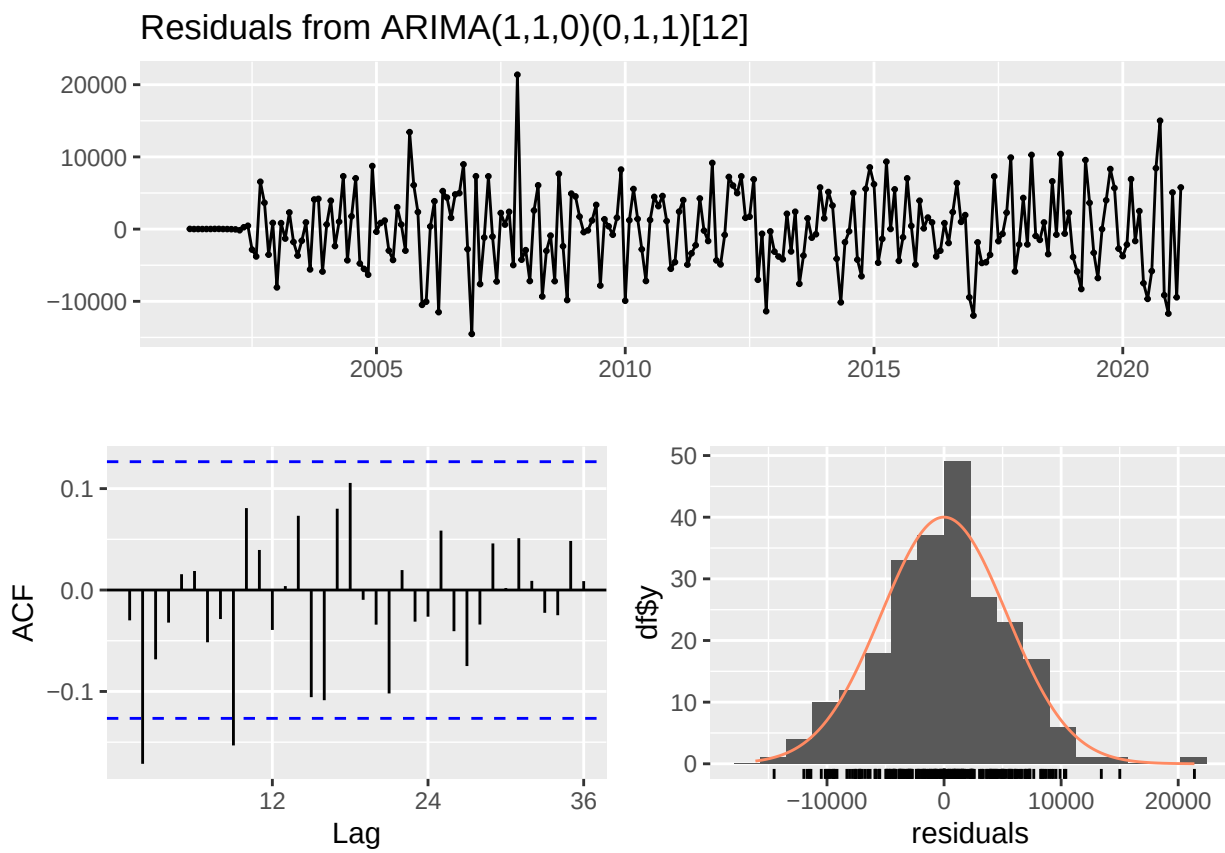
```r
Acf(gas_110010$residuals)
```




```
Pacf(gas_110010$residuals)
```

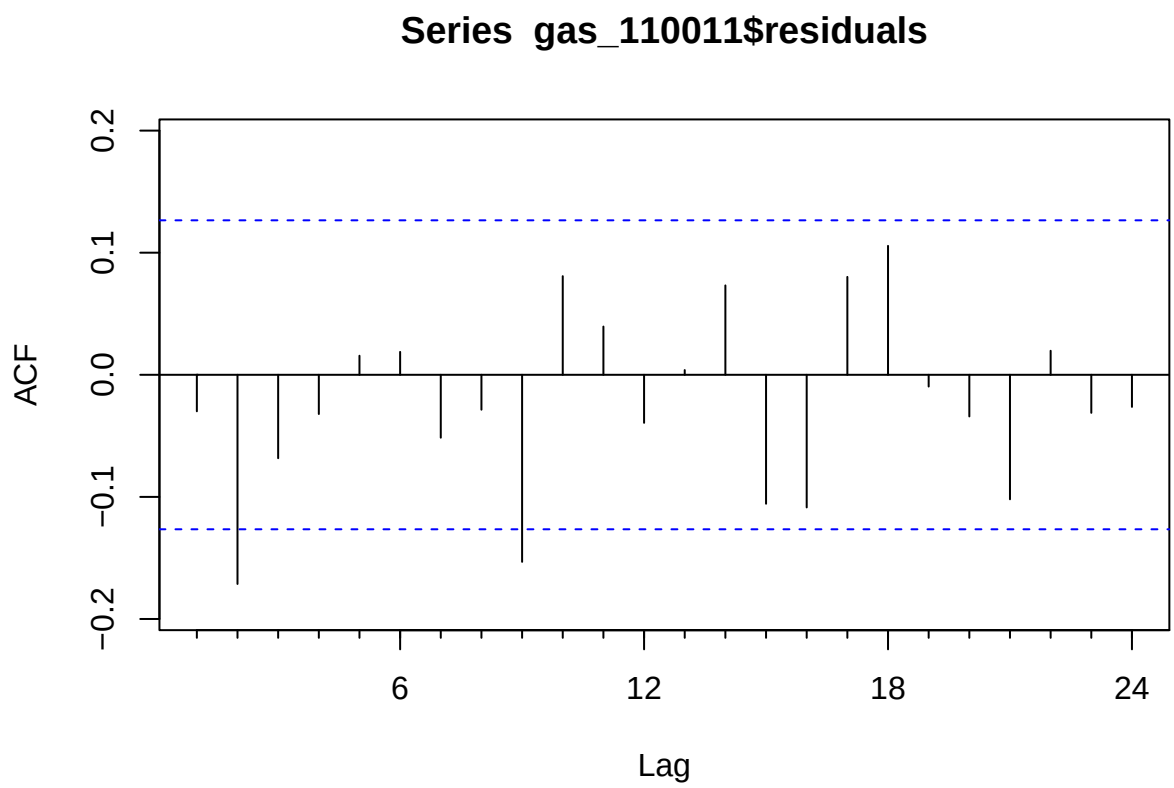



```
# moderately strong negative at lag 12 and weak negative at lag 24 - supports adding Q = 1

gas_110011 <- Arima(gas_ts, order = c(1,1,0), seasonal = c(0,1,1))
checkresiduals(gas_110011)
```

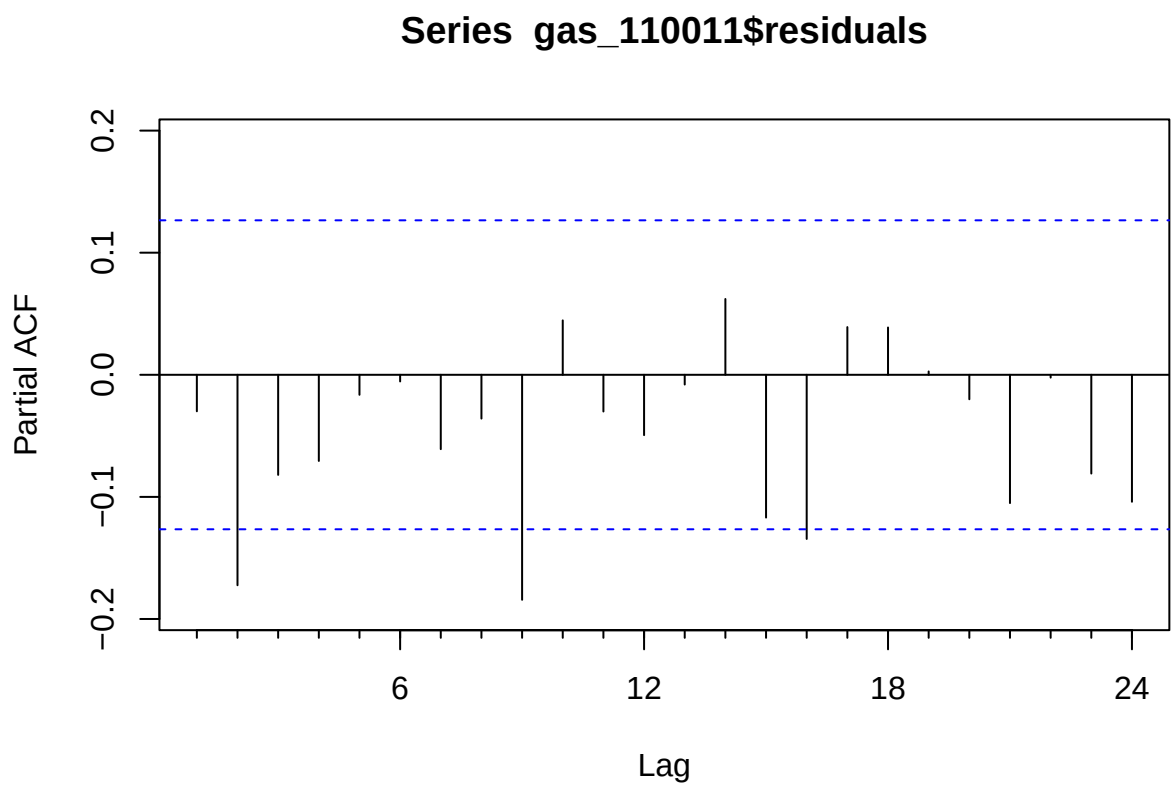



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,0)(0,1,1)[12]
## Q* = 33.681, df = 22, p-value = 0.05291
##
## Model df: 2.   Total lags used: 24
```

```r
Acf(gas_110011$residuals)
```




```
Pacf(gas_110011$residuals)
```

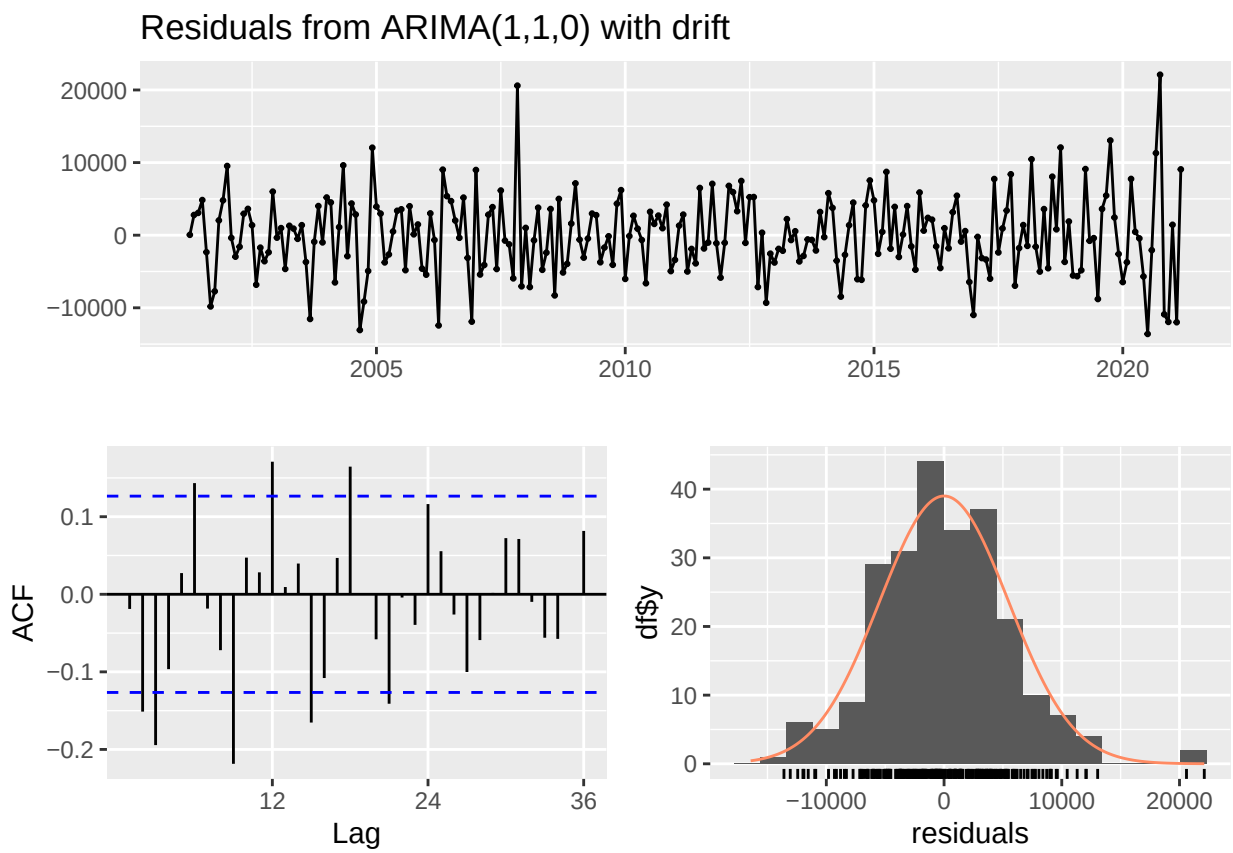



```
# ACF & PACF are neat & tidy now - no significant lags!

compare_new_aic <- data.frame(gas_110$aic, gas_110010$aic, gas_110011$aic)
# AIC supports our conclusion that SARIMA(1,1,0)(0,1,1) is an improvement!
```

Q8

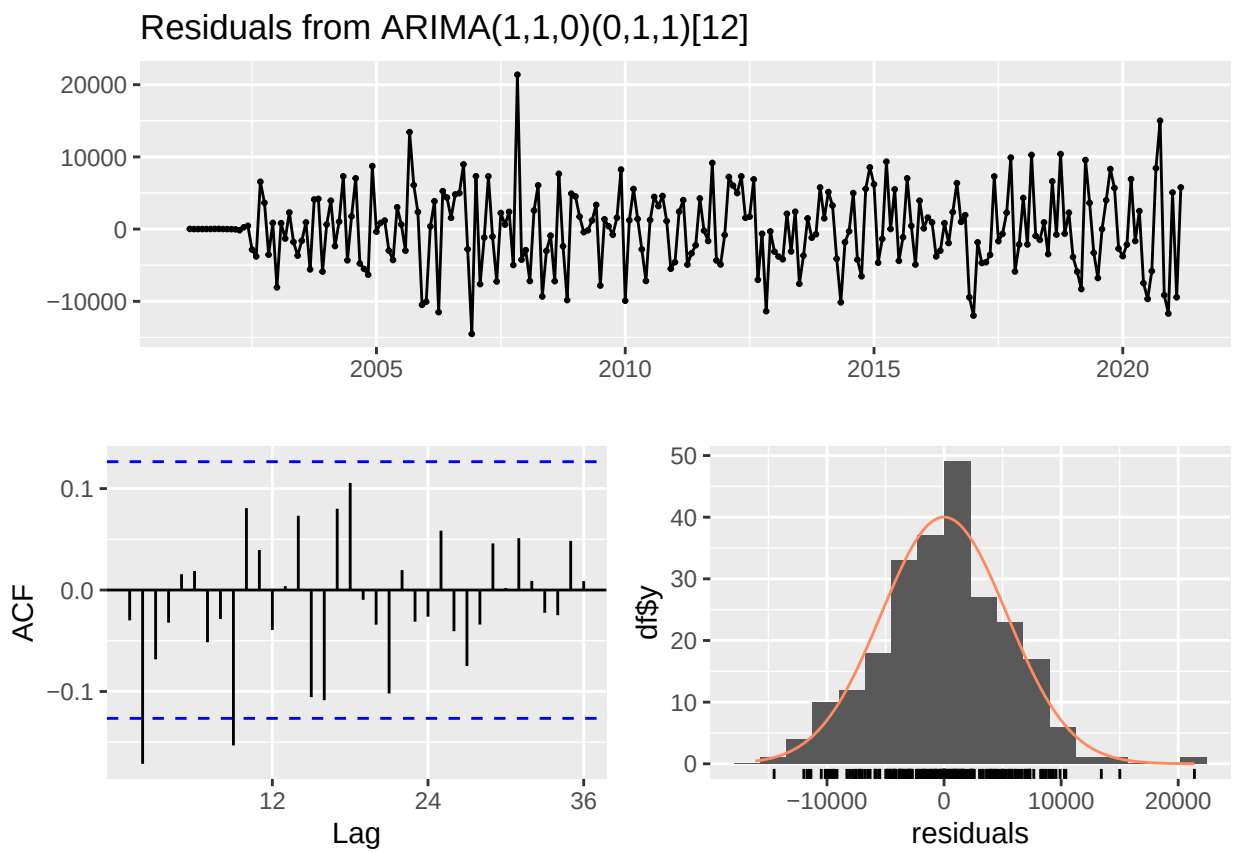
Compare the residual series for Q7 and Q6. Can you tell which ARIMA model is better representing the Natural Gas Series? Is that a fair comparison? Explain your response.

```
checkresiduals(gas_110)
```



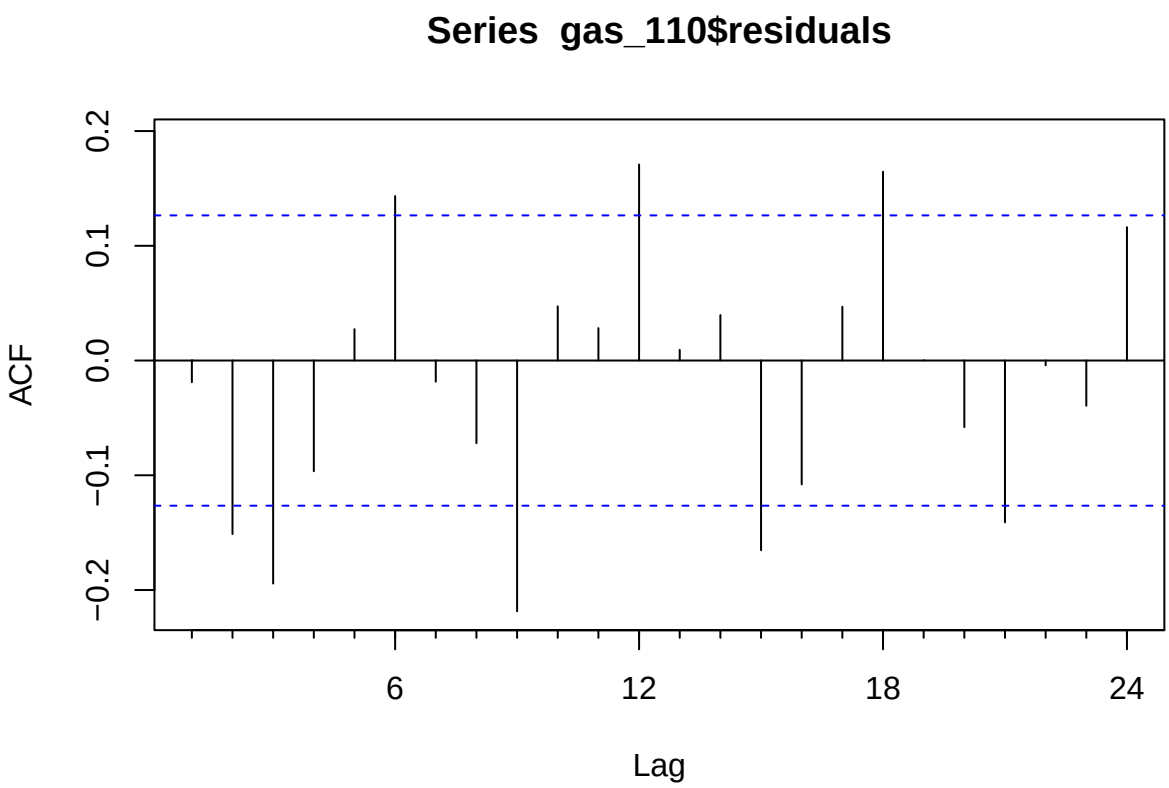

```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,0) with drift
## Q* = 72.475, df = 23, p-value = 5.015e-07
##
## Model df: 1.   Total lags used: 24
```

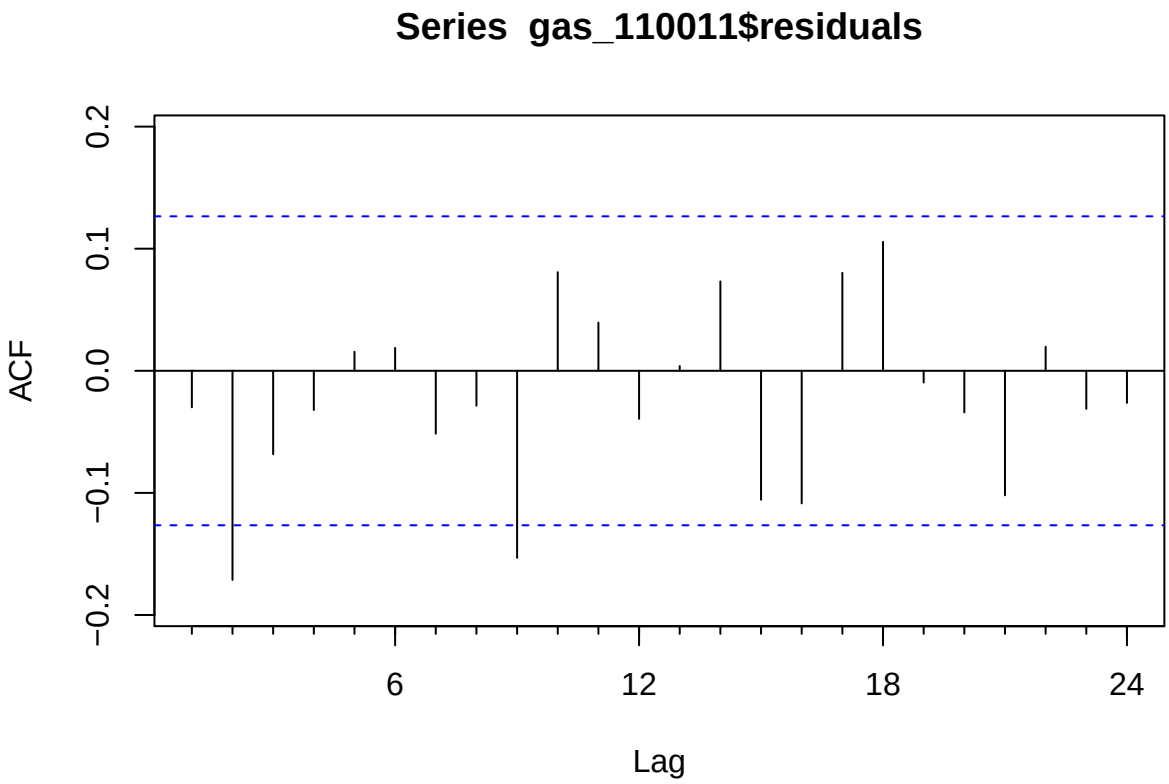
```r
checkresiduals(gas_110011)
```

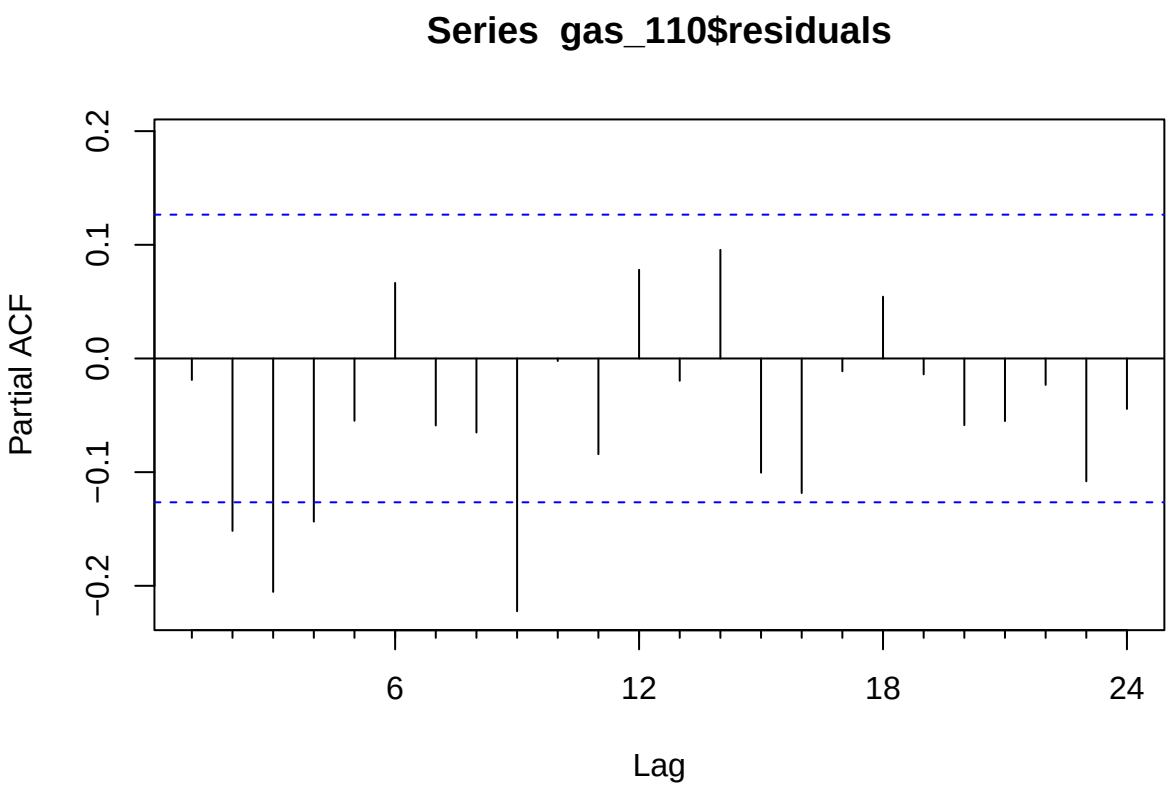


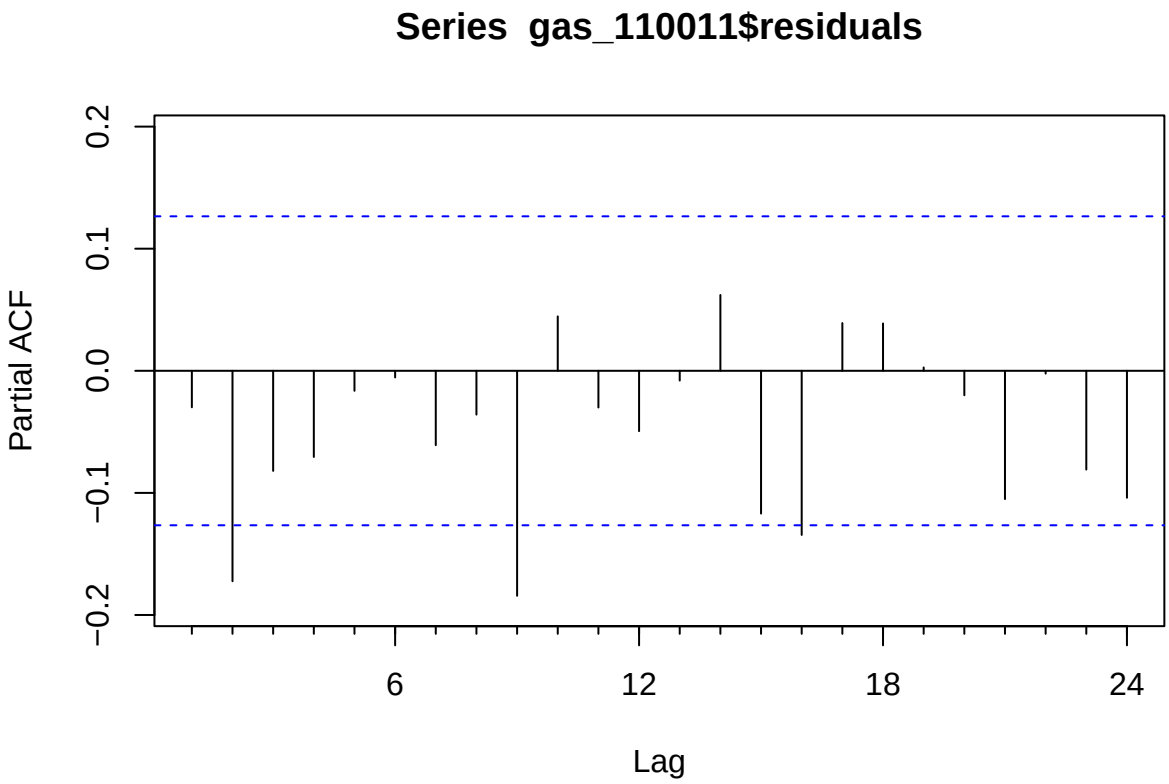

```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,0)(0,1,1)[12]
## Q* = 33.681, df = 22, p-value = 0.05291
##
## Model df: 2.   Total lags used: 24
```

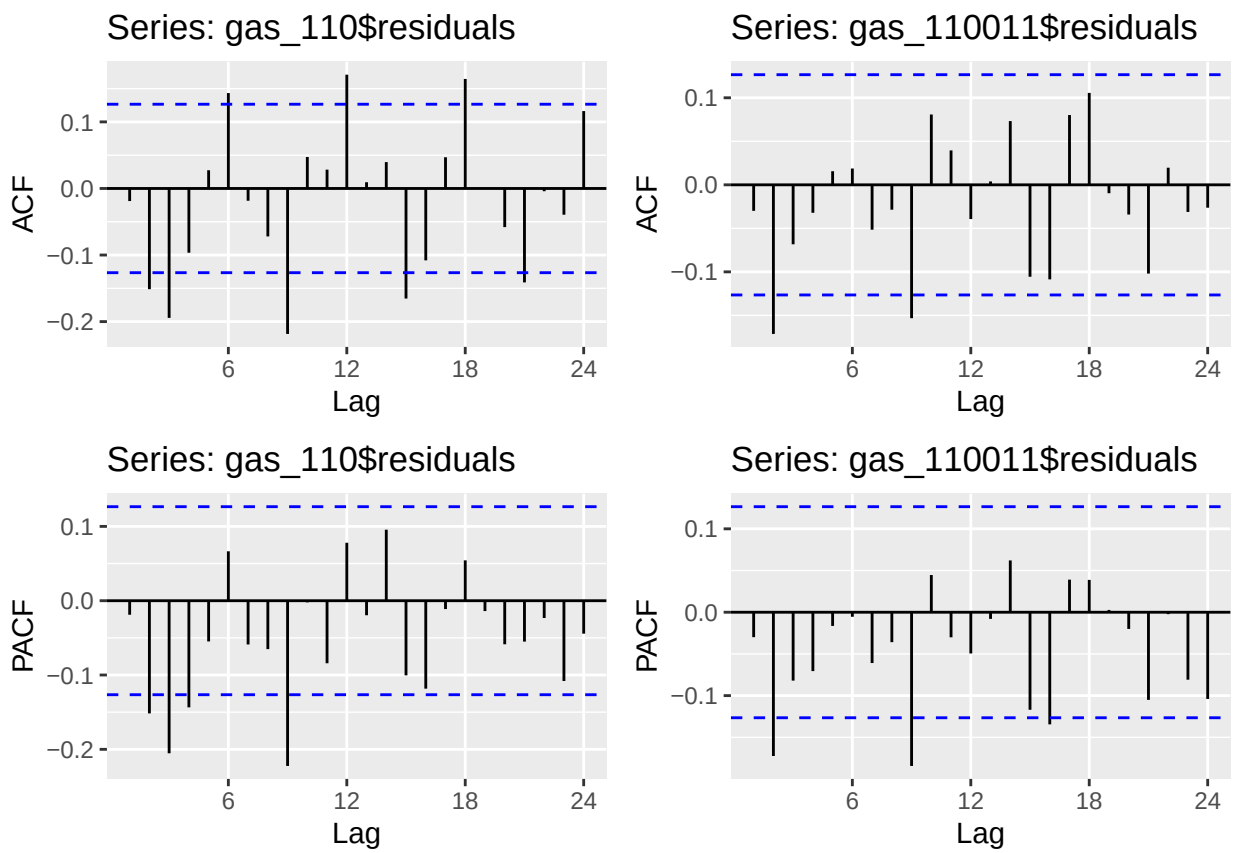
```
cowplot::plot_grid(autoplot(Acf(gas_110$residuals)), autoplot(Acf(gas_110011$residuals)),
                   autoplot(Pacf(gas_110$residuals)),autoplot(Pacf(gas_110011$residuals)), ncol = 2, nr
```


Series gas_110$residuals
ACF
0.2
0.1
0.0
−0.1
−0.2
6
12
18
24
Lag


Series gas_110011$residuals
ACF
0.2
0.1
0.0
−0.1
−0.2
6
12
18
24
Lag


Series gas_110$residuals
Partial ACF
0.2
0.1
0.0
−0.1
−0.2
6
12
18
24
Lag


Series gas_110011$residuals
Partial ACF
0.2
0.1
0.0
−0.1
−0.2
6
12
18
24
Lag



Answer: the ARIMA(1,1,0) residuals appear like white noise at first glance, but the ACF and PACF each have some lags that start to approach significance. On the other hand, ARIMA(1,1,0)(0,1,1) has ACF & PACF plots that generally look much neater and contained within the insignificance threshold. I'm slightly nervous that perhaps I've over-differenced the data with the ARIMA(1,1,0)(0,1,1), but I Think it's okay?

Checking your model with the auto.arima()

Please do not change your answers for Q4 and Q7 after you ran the *auto.arima*(). It is **ok** if you didn't get all orders correctly. You will not loose points for not having the same order as the *auto.arima*().

Q9

Use the *auto.arima*() command on the **deseasonalized series** to let R choose the model parameter for you. What's the order of the best ARIMA model? Does it match what you specified in Q4?

```
auto.arima(gas_deseas)
```

```
## Series: gas_deseas
## ARIMA(1,1,1) with drift
##
## Coefficients:
##          ar1      ma1     drift
##       0.7065  -0.9795  359.5052
## s.e.  0.0633   0.0326   29.5277
##
## sigma^2 = 26980609:  log likelihood = -2383.11
```

```
## AIC=4774.21   AICc=4774.38   BIC=4788.12
```

Answer: auto.arima indicates that ARIMA(1,1,1) with drift is the best model. I missed the Moving Average component!

Q10

Use the *auto.arima*() command on the **original series** to let R choose the model parameters for you. Does it match what you specified in Q7?

```r
auto.arima(gas_ts)
```

```
## Series: gas_ts
## ARIMA(1,0,0)(0,1,1)[12] with drift
##
## Coefficients:
##          ar1     sma1     drift
##       0.7416  -0.7026  358.7988
## s.e.  0.0442   0.0557   37.5875
##
## sigma^2 = 27569124:  log likelihood = -2279.54
## AIC=4567.08   AICc=4567.26   BIC=4580.8
```

answer: the auto.arima function has selected ARIMA(1,0,0)(0,1,1) with drift. It appears that I likely did actually over-difference the series. It's interesting that the seasonal component takes care of moving average part identified in Question 9.